Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Военный факультет

Дисциплина «Жизненный цикл разработки программного обеспечения»

ОТЧЕТ
по лабораторной работе №1
на тему

ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО «Утилита для расчета количества совершенных грехов»

Студент гр. 434701 А. А. Мороз

Преподаватель О. М. Внук

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Интегрированная среда разработки и паттерн MVC 4
1.1 Состав интегрированной среды разработки 4
1.2 Приемы работы в интегрированной среде разработки 5
1.3 Паттерн MVC. Активная и пассивная модель 5
2 Разработка приложения 8
2.1 Выбор языка программирования и среды разработки 8
2.2 Структура проекта 8
2.3 Реализация каркаса MVC с активной моделью 9
2.4 Алгоритм расчета количества грехов 10
3 Результаты работы программы 11
Заключение 14
Список использованных источников 15
Приложение А (обязательное) Листинг кода 16

ВВЕДЕНИЕ

Целью лабораторной работы является изучение методов работы с интегрированной средой разработки программного обеспечения, приобретение навыка разработки в выбранной среде приложения с графическим интерфейсом пользователя и изучение архитектурного паттерна MVC с активной и пассивной моделью.

Интегрированная среда разработки объединяет в одном приложении все инструменты, необходимые на протяжении жизненного цикла разработки программного обеспечения: редактор кода, компилятор, средства сборки и запуска, отладчик и средства интеграции с системой управления версиями. Применение архитектурного паттерна MVC на этапе проектирования обеспечивает разделение данных, логики их обработки и пользовательского интерфейса, что упрощает сопровождение и развитие программы на последующих этапах жизненного цикла.

Исходное задание по варианту 13: разработать утилиту для расчета количества совершенных грехов. На главной форме разместить кнопку «Рассчитать», в отображаемом окне разместить вопросы утилиты (например, дата рождения). Последние введенные данные запоминать в главной форме и восстанавливать при повторном вводе. Если введенные данные некорректны, показать окно с сообщением об этом. После этого на главной форме отобразить вычисленное количество грехов по своей формуле. Добавить каркас MVC с активной моделью [1].

Дополнительно требуется выбрать объектно-ориентированный язык программирования и среду разработки, установить и настроить программное обеспечение, ознакомиться с его функциональными возможностями и средствами отладки, создать проект под выданный вариант задания и описать разработанный каркас MVC.

1 ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ И ПАТТЕРН MVC

1.1 Состав интегрированной среды разработки

Интегрированная среда разработки, IDE (англ. Integrated Development Environment) – комплекс программных средств, используемых программистами для разработки программного обеспечения. Назначение среды состоит в том, чтобы объединить в одном приложении все инструменты, необходимые на протяжении жизненного цикла разработки: от написания исходного текста программы до ее отладки и передачи в систему управления версиями.

Большинство современных интегрированных сред разработки, как правило, предназначены для работы с несколькими языками программирования. К ним относятся IntelliJ IDEA, NetBeans, Eclipse, Qt Creator, Embarcadero RAD Studio, Code::Blocks, Xcode или Microsoft Visual Studio. Однако существуют и среды, ориентированные на один определенный язык программирования.

В состав интегрированной среды разработки входят редактор кода, компилятор и (или) интерпретатор, средства автоматизации сборки проекта, средства запуска проекта, отладчик, средства тестирования и средства поддержки совместной работы, то есть интеграция с системами управления версиями, и механизм расширений. Многие современные среды дополнительно включают браузер классов, инспектор объектов и диаграмму иерархии классов, используемые при объектно-ориентированной разработке, а также визуальный конструктор графического интерфейса пользователя. Состав среды разработки изображен на рисунке 1.1.

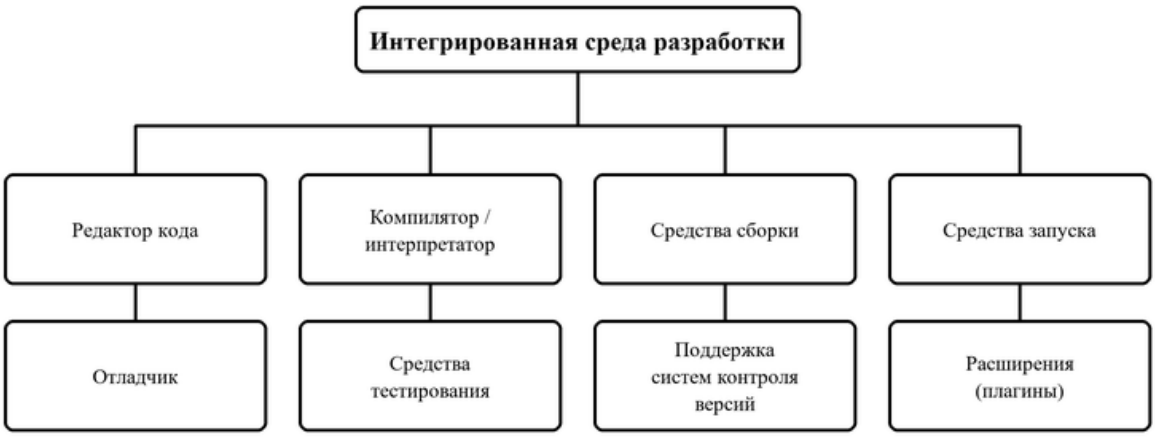


Рисунок 1.1 – Основные компоненты интегрированной среды разработки

1.2 Приемы работы в интегрированной среде разработки

Редактор кода является основным рабочим инструментом программиста. К основным приемам работы с ним относятся подсветка синтаксиса и автодополнение вводимых конструкций; навигация по коду – переход к определению или объявлению символа, переключение между файлом заголовка и файлом реализации, поиск символа по всему проекту; поиск и замена, в том числе с использованием регулярных выражений; рефакторинг – переименование символа по всему проекту, вынесение фрагмента в отдельную функцию, автоматическое создание определения метода по его объявлению; сворачивание блоков кода; автоматическое форматирование исходного кода и выравнивание отступов в соответствии с заданным стилем; использование шаблонов кода; работа с закладками; отображение ошибок и предупреждений компилятора непосредственно в тексте программы.

Сборка проекта выполняется средствами системы сборки, описание которой хранится в файле проекта. Процесс сборки состоит из нескольких последовательных этапов: предварительная обработка исходного текста, компиляция каждого модуля в объектный файл и компоновка объектных файлов с необходимыми библиотеками в исполняемый файл. Запуск проекта выполняется средой автоматически после успешной сборки.

Запуск программы под управлением отладчика позволяет устанавливать точки останова, выполнять программу по шагам с заходом в функции и без захода, наблюдать значения переменных и содержимое стека вызовов в момент останова. Отладчик является штатным средством поиска ошибок времени выполнения и применяется тогда, когда результат работы программы не соответствует ожидаемому, а причина по исходному тексту не очевидна.

1.3 Паттерн MVC. Активная и пассивная модель

MVC (Model-View-Controller») – архитектурный паттерн, разделяющий приложение на три компонента с разными зонами ответственности. Модель (Model) содержит данные приложения и бизнес-логику: хранение состояния, проверку корректности данных и вычисления; она не зависит от способа отображения информации. Представление (View) отвечает за отображение состояния модели пользователю и не содержит вычислительной логики. Контроллер (Controller) обрабатывает действия пользователя и преобразует их в вызовы методов модели.

Основной смысл такого разделения состоит в обеспечении независимости компонентов друг от друга: логику вычислений можно изменять, не затрагивая интерфейс, и наоборот; кроме того, одну и ту же модель может отображать несколько представлений одновременно.

Различают две разновидности паттерна – с пассивной и с активной моделью. Отличие между ними состоит в том, какой компонент сообщает представлению об изменении данных.

Пассивная модель об изменениях не оповещает: она лишь хранит данные и умеет их изменять. После вызова метода модели контроллер обязан явным образом уведомить представление и обновить его. Модель при этом ничего не знает о существовании подписчиков. Схема взаимодействия компонентов при пассивной модели изображена на рисунке 1.2.

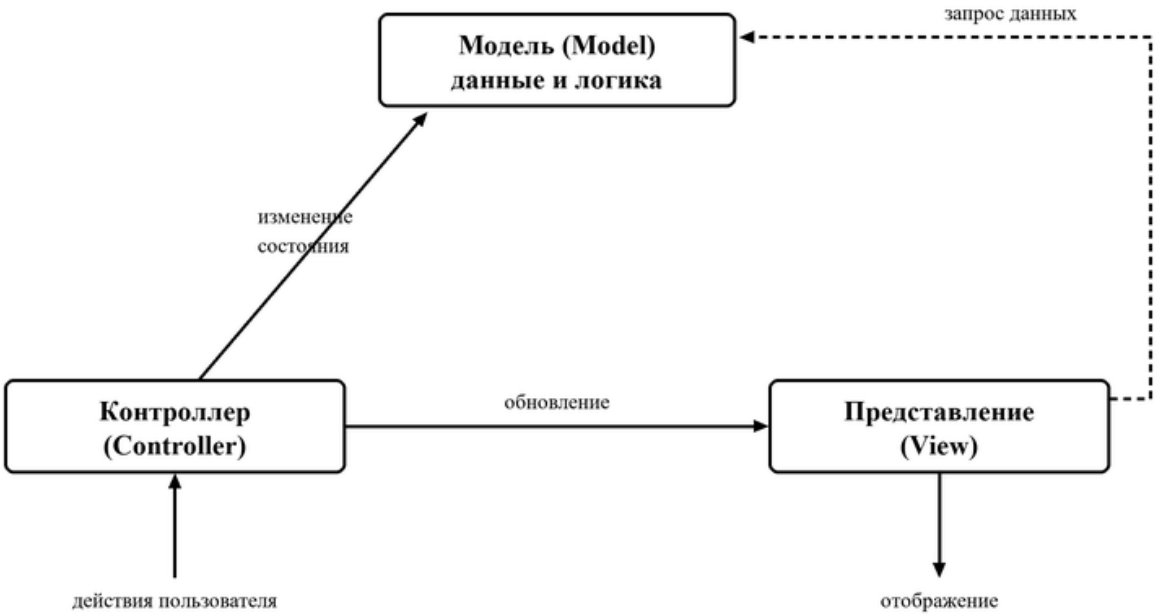


Рисунок 1.2 – Схема паттерна MVC с пассивной моделью

Активная модель реализует шаблон «Наблюдатель» (Observer). Она по-прежнему не знает о конкретных представлениях, однако при любом изменении своего состояния самостоятельно рассылает уведомление всем подписавшимся объектам. Представление подписывается на модель напрямую и обновляет себя без участия контроллера [2].

Схема взаимодействия компонентов при активной модели изображена на рисунке 1.3.

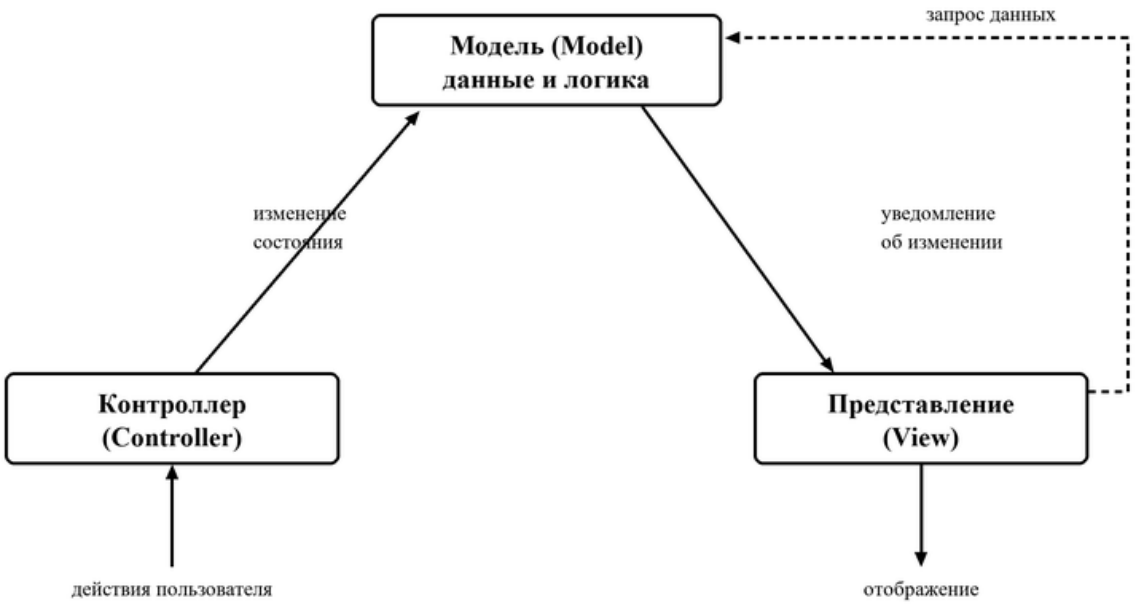


Рисунок 1.3 – Схема паттерна MVC с активной моделью

Активная модель применяется в тех случаях, когда состояние может измениться не только вследствие действий пользователя – например, по таймеру, по данным из сети или из фонового потока, – а также когда одну модель отображают несколько представлений: в этом случае все они синхронизируются автоматически. По варианту задания требуется реализовать каркас MVC именно с активной моделью.

2 РАЗРАБОТКА ПРИЛОЖЕНИЯ

2.1 Выбор языка программирования и среды разработки

В качестве основного языка программирования для реализации приложения выбран объектно-ориентированный язык C++, в качестве среды разработки – Qt Creator из состава Qt 6.11.2. Графический интерфейс приложения построен на библиотеке Qt Widgets. Сборка выполняется системой сборки qmake с использованием MinGW (GCC 13.1.0), входящего в комплект поставки Qt [3, 4].

Выбор данного инструментария обусловлен тем, что механизм сигналов и слотов, встроенный в Qt, представляет собой готовую реализацию шаблона «Наблюдатель», а именно он требуется для построения каркаса MVC с активной моделью по заданному варианту.

Проект создан как приложение Qt Widgets и описывается файлом проекта SinCalculator.pro. Заголовочные файлы классов перечислены в переменной HEADERS: по этому списку препроцессор moc обрабатывает классы, содержащие макрос Q_OBJECT, и формирует реализацию сигналов и слотов.

2.2 Структура проекта

Исходные тексты программы разделены по каталогам в соответствии с компонентами паттерна MVC: каталог src/model содержит модель, каталог src/view – классы представления, каталог src/controller – контроллер. Соответствие классов программы компонентам паттерна приведено в таблице 2.1.

Таблица 2.1 – Соответствие классов программы компонентам паттерна MVC

Компонент	Класс (файл)	Назначение
Модель	SinModel (src/model)	Хранение введенных данных и результата, проверка корректности данных, расчет количества грехов, рассылка уведомлений
Представление	MainWindow (src/view)	Главное окно: кнопка «Рассчитать», метки последних введенных данных и результата расчета, вывод сообщения об ошибке
Представление	InputDialog (src/view)	Модальное окно с вопросами утилиты, прием и выдача введенного текста
Контроллер	SinController (src/controller)	Обработка нажатия кнопки, открытие окна ввода, заполнение его данными модели, передача введенных значений в модель
Точка входа	main.cpp	Создание модели, представления и контроллера, связывание между собой, запуск цикла обработки событий

2.3 Реализация каркаса MVC с активной моделью

Модель реализована классом SinModel, унаследованным от QObject. Класс хранит последние введенные пользователем данные в структуре SinInput и результат расчета в структуре SinResult, выполняет проверку корректности введенных данных и вычисляет количество грехов. Класс модели не включает ни одного заголовочного файла представления или контроллера, то есть не знает о том, кто его использует.

Активность модели в разработанном приложении обеспечивается тремя сгиналами: inputChanged – введенные данные приняты моделью и запомнены; resultChanged – выполнен расчет; validationFailed – введенные данные некорректны. Главное окно подписывается на эти сигналы в своем конструкторе и обновляет себя самостоятельно, без участия контроллера.

Введенные данные хранятся в модели в виде строк, а не чисел. Это позволяет запоминать и восстанавливать именно то, что набрал пользователь, включая некорректный ввод, как требует условие задания. Разбор строк и проверка значений выполняются в модели, поскольку правила вида «год рождения не ранее 1900» и «дата не может быть в будущем» относятся к логике задачи, а не к свойствам элементов интерфейса. Благодаря этому окно ввода остается полностью пассивным и может быть заменено без изменения логики.

Контроллер реализован классом SinController. При нажатии кнопки «Рассчитать» он создает окно ввода, заполняет его последними данными, полученными от модели, и передает введенные пользователем значения в модель вызовом метода submit. Результат расчета контроллер не отображает и указателя на главное окно не имеет: обновление интерфейса выполняется по цепочке «модель – сигнал – слот представления».

Сообщение об ошибке выводится представлением, так как ошибка является частью состояния модели и приходит тем же путем, что и результат расчета.

Дата рождения вводится тремя раздельными текстовыми полями (день, месяц, год), поскольку по условию задания использование стандартного элемента управления для ввода даты не допускается. Корректность даты проверяется в модели, что позволяет отсеять несуществующие даты вида 31 февраля.

2.4 Алгоритм расчета количества грехов

Количество грехов вычисляется по формуле, состоящей из четырех шагов.

Шаг 1. Определяется число дней сознательной жизни как количество дней, прошедших с момента достижения человеком возраста семи лет. До этого возраста грехи не засчитываются, поэтому для детей младше семи лет результат равен нулю.

Шаг 2. Вычисляется греховная нагрузка за неделю как сумма произведений: количество случаев лжи с весом 1,0, количество случаев зависти с весом 0,8, количество случаев сквернословия с весом 0,5 и время, проведенное в социальных сетях, с весом 0,3 за час.

Шаг 3. Недельная нагрузка делится на семь и умножается на число дней сознательной жизни, после чего результат умножается на коэффициент искупления, зависящий от частоты покаяния: 1,0 – никогда, 0,9 – раз в год, 0,75 – раз в месяц, 0,5 – каждую неделю.

Шаг 4. Полученному числу ставится в соответствие словесная качественная оценка: 0 – «Безгрешен», менее 1000 – «Праведник», менее 10 000 – «Обычный грешник», менее 50 000 – «Заядлый грешник», иначе – «Пора срочно на исповедь».

Проверке подлежат все введенные значения: день от 1 до 31, месяц от 1 до 12, год от 1900 до текущего, существование даты в календаре, отсутствие даты в будущем, счетчики от 0 до 1000 целыми числами и время в социальных сетях от 0 до 24 часов.

3 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

После запуска программы отображается главное окно с кнопкой «Рассчитать». Значения исходных данных и результата не заполнены, вместо них выведены прочерки. Вид главного окна после запуска изображен на рисунке 3.1.

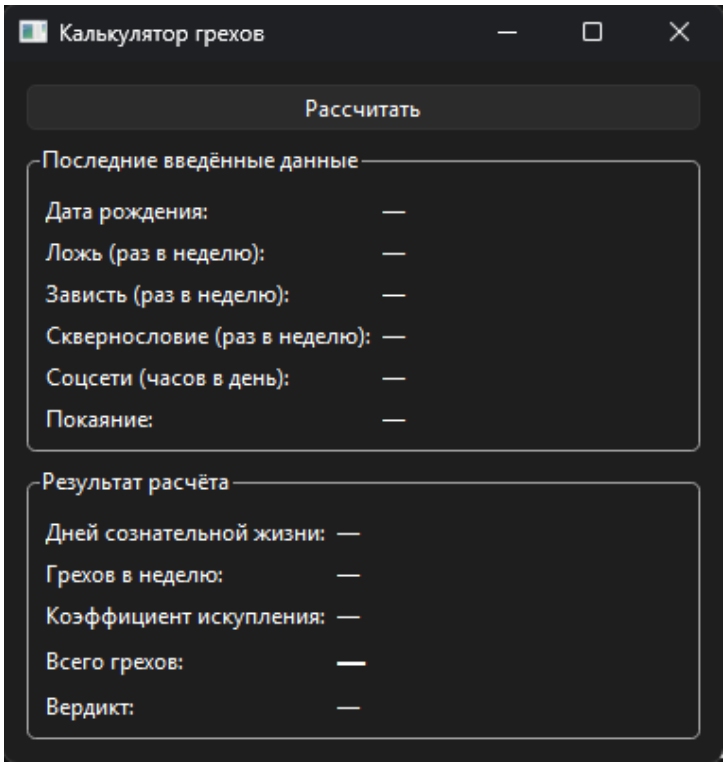


Рисунок 3.1 – Главное окно программы после запуска

По нажатию на кнопку «Рассчитать» открывается модальное окно с вопросами утилиты. Заполненное окно ввода изображено на рисунке 3.2.

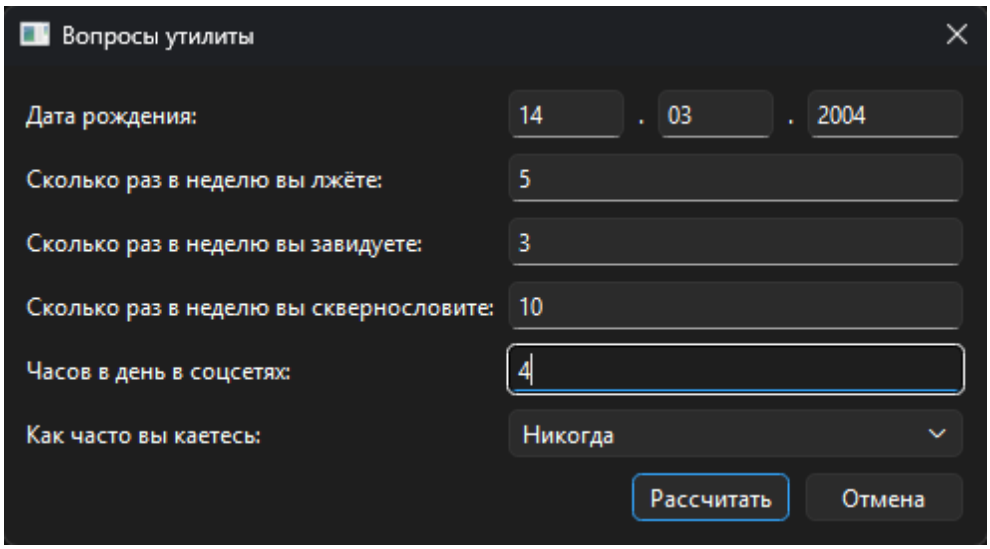


Рисунок 3.2 – Окно ввода исходных данных

После подтверждения ввода модель принимает данные, выполняет расчет и рассылает уведомления, по которым главное окно отображает введенные данные и результат. Главное окно с результатом расчета изображено на рисунке 3.3.

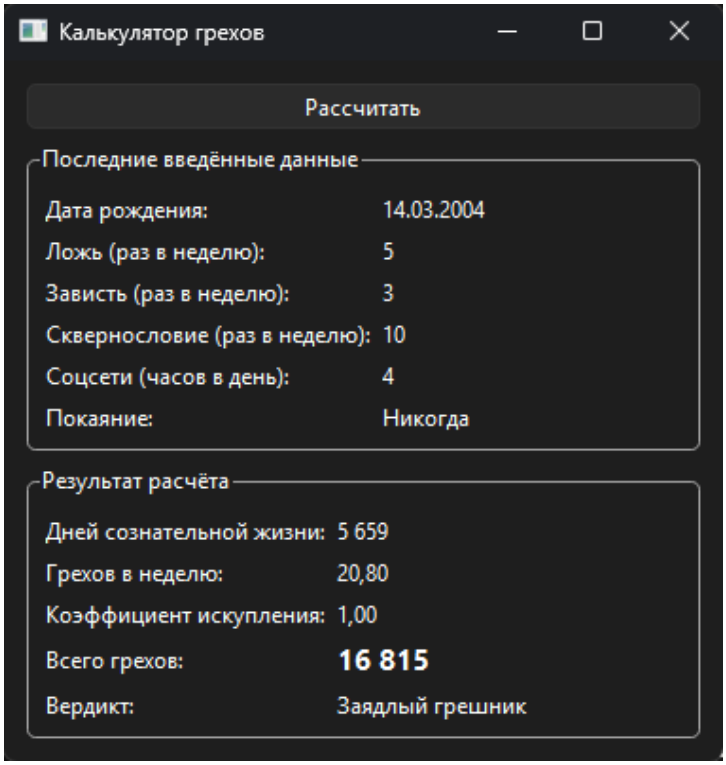


Рисунок 3.3 – Главное окно с результатом расчета

При повторном нажатии на кнопку «Рассчитать» окно ввода восстанавливает последние введенные значения, полученные из модели. Восстановленные данные изображены на рисунке 3.4.

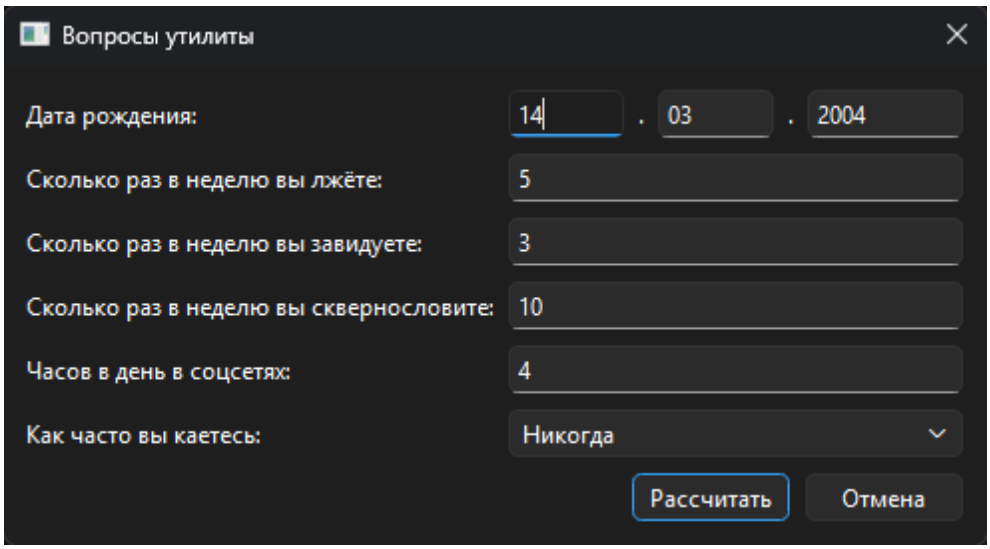


Рисунок 3.4 – Восстановление последних введенных данных

Если введенные данные некорректны, модель формирует сигнал об ошибке, по которому главное окно выводит сообщение с описанием причины и сбрасывает устаревший результат. Сообщение об ошибке при вводе несуществующего дня изображено на рисунке 3.5.

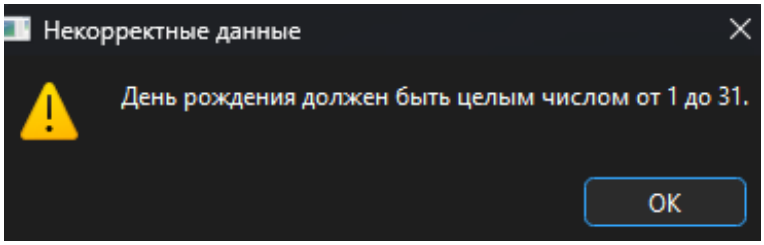


Рисунок 3.5 – Сообщение о некорректных данных

Все элементы задания по варианту 13 выполнены: главная форма содержит кнопку «Рассчитать», вопросы утилиты вынесены в отдельное окно, последние введенные данные запоминаются и восстанавливаются при повторном вводе, при некорректных данных выводится сообщение об ошибке, вычисленное количество грехов отображается на главной форме, каркас MVC построен с активной моделью.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены назначение, состав и основные приемы работы с интегрированной средой разработки программного обеспечения, а также архитектурный паттерн MVC и принципиальные различия между его реализациями с пассивной и активной моделью.

В качестве среды разработки выбран Qt Creator, в качестве языка программирования – C++ с библиотекой Qt Widgets. В выбранной среде создан проект и разработано приложение с графическим интерфейсом пользователя, реализующее задание варианта 13 – утилиту для расчета количества совершенных грехов.

Каркас приложения построен в соответствии с архитектурным паттерном MVC с использованием активной модели: класс SinModel хранит данные и выполняет вычисления, самостоятельно оповещая подписчиков сигналами; классы MainWindow и InputDialog образуют представление и подписываются на модель напрямую; класс SinController обрабатывает действия пользователя и не участвует в обновлении интерфейса. Такое разделение делает логику расчета независимой от способа отображения данных.

Работоспособность приложения подтверждена: реализованы запоминание и восстановление последних введенных данных, проверка корректности ввода с выводом сообщения об ошибке и отображение результата расчета на главной форме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Жизненный цикл разработки программного обеспечения. Лабораторная работа №1. Интегрированная среда разработки ПО [Электронный ресурс] : задание к лабораторной работе. – Минск : БГУИР, 2025.

[2] Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – СПб. : Питер, 2021. – 448 с.

[3] Qt Documentation [Электронный ресурс] / The Qt Company. – Режим доступа : https://doc.qt.io/.

[4] Qt Creator Manual [Электронный ресурс] / The Qt Company. – Режим доступа : https://doc.qt.io/qtcreator/.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода

Файл проекта SinCalculator.pro

```
QT       += core gui widgets

TEMPLATE  = app
TARGET    = SinCalculator

CONFIG   += c++17

# Заголовки ищутся от каталога src: "model/sinmodel.h", "view/mainwindow.h" и
т.д.
INCLUDEPATH += src

SOURCES += \
    src/main.cpp \
    src/model/sinmodel.cpp \
    src/view/mainwindow.cpp \
    src/view/inputdialog.cpp \
    src/controller/sincontroller.cpp

HEADERS += \
    src/model/sinmodel.h \
    src/view/mainwindow.h \
    src/view/inputdialog.h \
    src/controller/sincontroller.h
```

Файл src/model/sinmodel.h

```
#ifndef SINMODEL_H
#define SINMODEL_H

#include <QDate>
#include <QObject>
#include <QString>
#include <QStringList>

// Сырые данные, введённые пользователем в окне вопросов.
// Хранятся строками: проверку корректности выполняет модель, а не View.
struct SinInput
{
    QString day;
    QString month;
    QString year;
    QString lies;         // сколько раз в неделю лжёте
    QString envy;         // сколько раз в неделю завидуете
    QString swearing;     // сколько раз в неделю сквернословите
    QString socialHours;  // часов в день в соцсетях (праздность)
    int     penance = 0;  // индекс варианта покаяния
};

// Результат расчёта по формуле.
```

```
struct SinResult
{
    bool    valid          = false;
    qint64  consciousDays  = 0;    // дней сознательной жизни (с 7 лет)
    double  sinsPerWeek    = 0.0;  // греховная нагрузка за неделю
    double  redemption     = 1.0;  // коэффициент искупления
    qint64  totalSins      = 0;    // итоговое количество грехов
    QString verdict;               // словесная оценка
};

// АКТИВНАЯ модель: не знает ни о View, ни о контроллере.
// О любом своём изменении сама оповещает подписчиков сигналами.
class SinModel : public QObject
{
    Q_OBJECT

public:
    explicit SinModel(QObject *parent = nullptr);

    const SinInput  &input() const  { return m_input; }
    const SinResult &result() const { return m_result; }

    static QStringList penanceOptions();

public slots:
    // Единственная точка входа для контроллера.
    void submit(const SinInput &input);

signals:
    void inputChanged(const SinInput &input);
    void resultChanged(const SinResult &result);
    void validationFailed(const QString &message);

private:
    SinResult calculate(const QDate &birthDate,
                        int lies, int envy, int swearing,
                        double socialHours, int penance) const;

    SinInput  m_input;
    SinResult m_result;
};

#endif // SINMODEL_H
```

Файл src/model/sinmodel.cpp

```
#include "model/sinmodel.h"

#include <QtGlobal>
#include <cmath>

namespace {

// Возраст наступления «сознательности»: до 7 лет грехи не засчитываются.
constexpr int kAgeOfReason = 7;
```

```
// Веса грехов в формуле.
constexpr double kWeightLie      = 1.0;
constexpr double kWeightEnvy     = 0.8;
constexpr double kWeightSwearing = 0.5;
constexpr double kWeightIdleHour = 0.3;

// Коэффициенты искупления, соответствуют вариантам покаяния.
const double kRedemption[] = { 1.0, 0.9, 0.75, 0.5 };

// Разбор целого неотрицательного числа с проверкой диапазона.
bool parseInt(const QString &text, int minValue, int maxValue, int &out)
{
    bool ok = false;
    const int value = text.trimmed().toInt(&ok);
    if (!ok || value < minValue || value > maxValue)
        return false;
    out = value;
    return true;
}

// Разбор вещественного числа (запятая допускается как разделитель).
bool parseDouble(const QString &text, double minValue, double maxValue,
double &out)
{
    bool ok = false;
    QString normalized = text.trimmed();
    normalized.replace(QLatin1Char(','), QLatin1Char('.'));
    const double value = normalized.toDouble(&ok);
    if (!ok || value < minValue || value > maxValue)
        return false;
    out = value;
    return true;
}

} // namespace

SinModel::SinModel(QObject *parent)
    : QObject(parent)
{
}

QStringList SinModel::penanceOptions()
{
    return { QStringLiteral("Никогда"),
             QStringLiteral("Раз в год"),
             QStringLiteral("Раз в месяц"),
             QStringLiteral("Каждую неделю") };
}

void SinModel::submit(const SinInput &input)
{
    // Введённые данные запоминаются в любом случае, даже если они
    // некорректны: при повторном вводе окно будет ими восстановлено.
    m_input = input;
    emit inputChanged(m_input);

    int day = 0;
    int month = 0;
    int year = 0;
    const int currentYear = QDate::currentDate().year();
```

```
    if (!parseInt(input.day, 1, 31, day)) {
        emit validationFailed(QStringLiteral("День рождения должен быть целым
числом от 1 до 31."));
        return;
    }
    if (!parseInt(input.month, 1, 12, month)) {
        emit validationFailed(QStringLiteral("Месяц рождения должен быть
целым числом от 1 до 12."));
        return;
    }
    if (!parseInt(input.year, 1900, currentYear, year)) {
        emit validationFailed(QStringLiteral("Год рождения должен быть целым
числом от 1900 до %1.").arg(currentYear));
        return;
    }

    const QDate birthDate(year, month, day);
    if (!birthDate.isValid()) {
        emit validationFailed(QStringLiteral("Такой даты не существует в
календаре."));
        return;
    }
    if (birthDate > QDate::currentDate()) {
        emit validationFailed(QStringLiteral("Дата рождения не может быть в
будущем."));
        return;
    }

    int lies = 0;
    int envy = 0;
    int swearing = 0;
    double socialHours = 0.0;

    if (!parseInt(input.lies, 0, 1000, lies)) {
        emit validationFailed(QStringLiteral("Количество случаев лжи за
неделю — целое число от 0 до 1000."));
        return;
    }
    if (!parseInt(input.envy, 0, 1000, envy)) {
        emit validationFailed(QStringLiteral("Количество случаев зависти за
неделю — целое число от 0 до 1000."));
        return;
    }
    if (!parseInt(input.swearing, 0, 1000, swearing)) {
        emit validationFailed(QStringLiteral("Количество бранных слов за
неделю — целое число от 0 до 1000."));
        return;
    }
    if (!parseDouble(input.socialHours, 0.0, 24.0, socialHours)) {
        emit validationFailed(QStringLiteral("Время в соцсетях — число от 0
до 24 часов в сутки."));
        return;
    }

    const int penanceCount = static_cast<int>(sizeof(kRedemption) /
sizeof(kRedemption[0]));
    if (input.penance < 0 || input.penance >= penanceCount) {
        emit validationFailed(QStringLiteral("Не выбрана частота
покаяния."));
        return;
    }
```

```
    m_result = calculate(birthDate, lies, envy, swearing, socialHours,
input.penance);
    emit resultChanged(m_result);
}

SinResult SinModel::calculate(const QDate &birthDate,
                              int lies, int envy, int swearing,
                              double socialHours, int penance) const
{
    SinResult result;
    result.valid = true;

    // 1. Сколько дней человек грешит осознанно.
    const qint64 days =
birthDate.addYears(kAgeOfReason).daysTo(QDate::currentDate());
    result.consciousDays = qMax<qint64>(0, days);

    // 2. Греховная нагрузка за одну неделю.
    result.sinsPerWeek = lies * kWeightLie
                       + envy * kWeightEnvy
                       + swearing * kWeightSwearing
                       + socialHours * 7.0 * kWeightIdleHour;

    // 3. Искупление уменьшает итог.
    result.redemption = kRedemption[penance];

    // 4. Итоговое количество грехов.
    const double total = result.sinsPerWeek / 7.0 *
static_cast<double>(result.consciousDays) * result.redemption;
    result.totalSins = static_cast<qint64>(std::llround(total));

    // 5. Словесная оценка.
    if (result.totalSins == 0)
        result.verdict = QStringLiteral("Безгрешен");
    else if (result.totalSins < 1000)
        result.verdict = QStringLiteral("Праведник");
    else if (result.totalSins < 10000)
        result.verdict = QStringLiteral("Обычный грешник");
    else if (result.totalSins < 50000)
        result.verdict = QStringLiteral("Заядлый грешник");
    else
        result.verdict = QStringLiteral("Пора срочно на исповедь");

    return result;
}
```

Файл src/view/mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QWidget>

#include "model/sinmodel.h"
```

```
class QLabel;

// Главное окно. В активной модели View подписывается на сигналы модели
// напрямую и обновляет себя сам, без участия контроллера.
class MainWindow : public QWidget
{
    Q_OBJECT

public:
    explicit MainWindow(SinModel *model, QWidget *parent = nullptr);

signals:
    // Намерение пользователя. Обрабатывает контроллер.
    void calculateRequested();

private slots:
    void showInput(const SinInput &input);
    void showResult(const SinResult &result);
    void showError(const QString &message);

private:
    SinModel *m_model;

    QLabel *m_birthDate;
    QLabel *m_lies;
    QLabel *m_envy;
    QLabel *m_swearing;
    QLabel *m_socialHours;
    QLabel *m_penance;

    QLabel *m_consciousDays;
    QLabel *m_sinsPerWeek;
    QLabel *m_redemption;
    QLabel *m_totalSins;
    QLabel *m_verdict;
};

#endif // MAINWINDOW_H
```

Файл src/view/mainwindow.cpp

```
#include "view/mainwindow.h"

#include <QFont>
#include <QFormLayout>
#include <QGroupBox>
#include <QLabel>
#include <QLocale>
#include <QMessageBox>
#include <QPushButton>
#include <QVBoxLayout>

namespace {

const QString kEmpty = QStringLiteral("—");
```

```
QLabel *makeValueLabel(QWidget *parent)
{
    auto *label = new QLabel(kEmpty, parent);
    label->setTextInteractionFlags(Qt::TextSelectableByMouse);
    return label;
}

} // namespace

MainWindow::MainWindow(SinModel *model, QWidget *parent)
    : QWidget(parent)
    , m_model(model)
    , m_birthDate(makeValueLabel(this))
    , m_lies(makeValueLabel(this))
    , m_envy(makeValueLabel(this))
    , m_swearing(makeValueLabel(this))
    , m_socialHours(makeValueLabel(this))
    , m_penance(makeValueLabel(this))
    , m_consciousDays(makeValueLabel(this))
    , m_sinsPerWeek(makeValueLabel(this))
    , m_redemption(makeValueLabel(this))
    , m_totalSins(makeValueLabel(this))
    , m_verdict(makeValueLabel(this))
{
    setWindowTitle(QStringLiteral("Калькулятор грехов"));

    auto *inputBox = new QGroupBox(QStringLiteral("Последние введённые
данные"), this);
    auto *inputForm = new QFormLayout(inputBox);
    inputForm->addRow(QStringLiteral("Дата рождения:"), m_birthDate);
    inputForm->addRow(QStringLiteral("Ложь (раз в неделю):"), m_lies);
    inputForm->addRow(QStringLiteral("Зависть (раз в неделю):"), m_envy);
    inputForm->addRow(QStringLiteral("Сквернословие (раз в неделю):"),
m_swearing);
    inputForm->addRow(QStringLiteral("Соцсети (часов в день):"),
m_socialHours);
    inputForm->addRow(QStringLiteral("Покаяние:"), m_penance);

    auto *resultBox = new QGroupBox(QStringLiteral("Результат расчёта"),
this);
    auto *resultForm = new QFormLayout(resultBox);
    resultForm->addRow(QStringLiteral("Дней сознательной жизни:"),
m_consciousDays);
    resultForm->addRow(QStringLiteral("Грехов в неделю:"), m_sinsPerWeek);
    resultForm->addRow(QStringLiteral("Коэффициент искупления:"),
m_redemption);
    resultForm->addRow(QStringLiteral("Всего грехов:"), m_totalSins);
    resultForm->addRow(QStringLiteral("Вердикт:"), m_verdict);

    QFont totalFont = m_totalSins->font();
    totalFont.setBold(true);
    totalFont.setPointSize(totalFont.pointSize() + 2);
    m_totalSins->setFont(totalFont);

    auto *calculateButton = new QPushButton(QStringLiteral("Рассчитать"),
this);
    connect(calculateButton, &QPushButton::clicked, this,
&MainWindow::calculateRequested);

    auto *layout = new QVBoxLayout(this);
    layout->addWidget(calculateButton);
```

```
    layout->addWidget(inputBox);
    layout->addWidget(resultBox);
    layout->addStretch();

    // Подписка View на активную модель.
    connect(m_model, &SinModel::inputChanged, this, &MainWindow::showInput);
    connect(m_model, &SinModel::resultChanged, this,
&MainWindow::showResult);
    connect(m_model, &SinModel::validationFailed, this,
&MainWindow::showError);

    setMinimumWidth(380);
}

void MainWindow::showInput(const SinInput &input)
{
    m_birthDate->setText(QStringLiteral("%1.%2.%3").arg(input.day,
input.month, input.year));
    m_lies->setText(input.lies);
    m_envy->setText(input.envy);
    m_swearing->setText(input.swearing);
    m_socialHours->setText(input.socialHours);
    m_penance->setText(SinModel::penanceOptions().value(input.penance,
kEmpty));
}

void MainWindow::showResult(const SinResult &result)
{
    const QLocale locale;
    m_consciousDays->setText(locale.toString(result.consciousDays));
    m_sinsPerWeek->setText(locale.toString(result.sinsPerWeek, 'f', 2));
    m_redemption->setText(locale.toString(result.redemption, 'f', 2));
    m_totalSins->setText(locale.toString(result.totalSins));
    m_verdict->setText(result.verdict);
}

void MainWindow::showError(const QString &message)
{
    // Прошлый результат перестал соответствовать введённым данным.
    m_consciousDays->setText(kEmpty);
    m_sinsPerWeek->setText(kEmpty);
    m_redemption->setText(kEmpty);
    m_totalSins->setText(kEmpty);
    m_verdict->setText(kEmpty);

    QMessageBox::warning(this, QStringLiteral("Некорректные данные"),
message);
}
```

Файл src/view/inputdialog.h

```
#ifndef INPUTDIALOG_H
#define INPUTDIALOG_H

#include <QDialog>

#include "model/sinmodel.h"
```

```
class QComboBox;
class QLineEdit;

// Отдельное окно с вопросами утилиты.
// Ничего не вычисляет и не проверяет — только собирает введённый текст.
class InputDialog : public QDialog
{
    Q_OBJECT

public:
    explicit InputDialog(QWidget *parent = nullptr);

    void setInput(const SinInput &input); // восстановление прошлого ввода
    SinInput input() const;               // выдача введённого контроллеру

private:
    QLineEdit *m_day;
    QLineEdit *m_month;
    QLineEdit *m_year;
    QLineEdit *m_lies;
    QLineEdit *m_envy;
    QLineEdit *m_swearing;
    QLineEdit *m_socialHours;
    QComboBox *m_penance;
};

#endif // INPUTDIALOG_H
```

Файл src/view/inputdialog.cpp

```
#include "view/inputdialog.h"

#include <QComboBox>
#include <QDialogButtonBox>
#include <QFormLayout>
#include <QHBoxLayout>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QVBoxLayout>

InputDialog::InputDialog(QWidget *parent)
    : QDialog(parent)
    , m_day(new QLineEdit(this))
    , m_month(new QLineEdit(this))
    , m_year(new QLineEdit(this))
    , m_lies(new QLineEdit(this))
    , m_envy(new QLineEdit(this))
    , m_swearing(new QLineEdit(this))
    , m_socialHours(new QLineEdit(this))
    , m_penance(new QComboBox(this))
{
    setWindowTitle(QStringLiteral("Вопросы утилиты"));
    setModal(true);

    // Дата рождения вводится тремя отдельными полями:
```

```
    // стандартный элемент ввода даты по условию использовать нельзя.
    m_day->setPlaceholderText(QStringLiteral("дд"));
    m_month->setPlaceholderText(QStringLiteral("мм"));
    m_year->setPlaceholderText(QStringLiteral("гггг"));
    m_day->setMaximumWidth(60);
    m_month->setMaximumWidth(60);
    m_year->setMaximumWidth(80);

    auto *dateLayout = new QHBoxLayout;
    dateLayout->addWidget(m_day);
    dateLayout->addWidget(new QLabel(QStringLiteral("."), this));
    dateLayout->addWidget(m_month);
    dateLayout->addWidget(new QLabel(QStringLiteral("."), this));
    dateLayout->addWidget(m_year);
    dateLayout->addStretch();

    m_penance->addItems(SinModel::penanceOptions());

    auto *form = new QFormLayout;
    form->addRow(QStringLiteral("Дата рождения:"), dateLayout);
    form->addRow(QStringLiteral("Сколько раз в неделю вы лжёте:"), m_lies);
    form->addRow(QStringLiteral("Сколько раз в неделю вы завидуете:"),
m_envy);
    form->addRow(QStringLiteral("Сколько раз в неделю вы сквернословите:"),
m_swearing);
    form->addRow(QStringLiteral("Часов в день в соцсетях:"), m_socialHours);
    form->addRow(QStringLiteral("Как часто вы каетесь:"), m_penance);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok |
QDialogButtonBox::Cancel, this);
    buttons->button(QDialogButtonBox::Ok)-
>setText(QStringLiteral("Рассчитать"));
    buttons->button(QDialogButtonBox::Cancel)-
>setText(QStringLiteral("Отмена"));
    connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);

    auto *layout = new QVBoxLayout(this);
    layout->addLayout(form);
    layout->addWidget(buttons);
}

void InputDialog::setInput(const SinInput &input)
{
    m_day->setText(input.day);
    m_month->setText(input.month);
    m_year->setText(input.year);
    m_lies->setText(input.lies);
    m_envy->setText(input.envy);
    m_swearing->setText(input.swearing);
    m_socialHours->setText(input.socialHours);
    m_penance->setCurrentIndex(input.penance);
}

SinInput InputDialog::input() const
{
    SinInput input;
    input.day         = m_day->text();
    input.month       = m_month->text();
    input.year        = m_year->text();
    input.lies        = m_lies->text();
```

```
    input.envy        = m_envy->text();
    input.swearing    = m_swearing->text();
    input.socialHours = m_socialHours->text();
    input.penance     = m_penance->currentIndex();
    return input;
}
```

Файл src/controller/sincontroller.h

```
#ifndef SINCONTROLLER_H
#define SINCONTROLLER_H

#include <QObject>

class QWidget;
class SinModel;

// Контроллер: принимает намерения пользователя, показывает окно ввода
// и передаёт собранные данные в модель. Результат не отображает —
// это делает View по сигналам активной модели.
class SinController : public QObject
{
    Q_OBJECT

public:
    SinController(SinModel *model, QWidget *dialogParent, QObject *parent =
nullptr);

public slots:
    void onCalculateRequested();

private:
    SinModel *m_model;
    QWidget  *m_dialogParent;
};

#endif // SINCONTROLLER_H
```

Файл src/controller/sincontroller.cpp

```
#include "controller/sincontroller.h"

#include "model/sinmodel.h"
#include "view/inputdialog.h"

SinController::SinController(SinModel *model, QWidget *dialogParent, QObject
*parent)
    : QObject(parent)
    , m_model(model)
    , m_dialogParent(dialogParent)
{
}

void SinController::onCalculateRequested()
```

```
{
    InputDialog dialog(m_dialogParent);

    // Восстановление последних введённых данных из модели.
    dialog.setInput(m_model->input());

    if (dialog.exec() != QDialog::Accepted)
        return;

    m_model->submit(dialog.input());
}
```

Файл src/main.cpp

```
#include <QApplication>

#include "controller/sincontroller.h"
#include "model/sinmodel.h"
#include "view/mainwindow.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    SinModel model;
    MainWindow window(&model);
    SinController controller(&model, &window);

    // Единственная связь View -> Controller: намерение пользователя.
    QObject::connect(&window, &MainWindow::calculateRequested,
                     &controller, &SinController::onCalculateRequested);

    window.show();
    return app.exec();
}
```